

Smali By bithowl: Chapter 5 Smali File Structure

("Every Smali File Tells a Story... If You Know How to Read It")



SMALI BY BITHOWL : CHAPTER 5

SMALI FILE STRUCTURE

"Every Smali File Tells a Story... If You Know How to Read It"

THE STORY (HOOK)

Imagine receiving a 500-page instruction manual. You don't know the language. You don't know the symbols. But every sentence points to another page. Every chapter references another chapter. Every object has an ID.

Instead of copying the same information repeatedly, DEX stores it once and references it using indexes.

💡 **Smali files follow a pattern. Learn the layout, and you can read any Smali file with confidence.**

A TYPICAL SMALI FILE

<code>.class</code>	Define the class
<code>.super</code>	Parent class
<code>.source</code>	Original source file
<code>.implements</code>	Implemented interfaces
<code>.field</code>	Class variables
<code>.method</code>	Method start
<code>...</code>	
<code>.end method</code>	Method end
<code>.method</code>	Method start
<code>...</code>	
<code>.end method</code>	Method end

Every directive describes one part of the compiled class.

WHAT IS A DEX FILE?

APK → **classes.dex** (DEX Bytecode) → ART Runtime

- ✓ Dalvik Executable format
- ✓ Generated from Java / Kotlin
- ✓ Compact, optimized & efficient
- ✓ Executed by Android Runtime (ART)
- ✓ Large apps may contain classes2.dex, classes3.dex, classes4.dex, etc.

⚡ **Everything Android needs to execute the app lives in the DEX file.**

1 .CLASS - DEFINING THE CLASS

```
.class public
Lcom/example/app/LoginActivity;
```

- This file represents a class.
- The class is public.
- Full name: com.example.app.LoginActivity

Type Descriptor Rules:

- L = Object Type
- / = replaces Java's ..
- ; = marks the end

```
Landroid/content/Intent;
= android.content.Intent
```

2 .SUPER - PARENT CLASS

```
.super Landroidx/appcompat/
app/ AppCompatActivity;
```

Equivalent Java:

```
extends AppCompatActivity
```

This tells Android which parent class should be inherited.

3 .SOURCE - SOURCE FILE

```
.source "LoginActivity.java"
```

Records the source filename used during compilation.

Obfuscation tools often rename or remove this information.

4 .IMPLEMENTS - INTERFACES

```
.implements Ljava/lang/Runnable;
```

Equivalent Java:

```
implements Runnable
```

A class may contain multiple .implements directives.

5 .FIELD - CLASS VARIABLES

```
.field private token:Ljava/lang/String;

Equivalent Java: private String token;

.field public static VERSION:I

Equivalent Java: public static int VERSION;
```

Modifiers:

- public • private • protected
- static • final • volatile • transient

Behaves similar to Java.

6 .METHOD - DEFINING A METHOD

```
.method public login()V

Equivalent Java: public void login()
```

Everything between:

```
.method
...
.end method
```

belongs to this method. Contains registers, instructions, method calls, branches, returns.

7 .END METHOD

```
.end method
```

Marks the end of the current method.

Like the closing brace {} in Java.

8 CONSTRUCTORS

Constructors use <init>

```
.method public <init>()V
```

Equivalent Java:

```
public LoginActivity()
```

Static initializer uses <clinit>

```
.method static <clinit>()V
```

Executed once when class is first loaded.

UNDERSTANDING METHODS (INSIDE)

```
...
.locals 2
const-string v0, "Hello"
invoke-virtual {p0, v0},
Landroid/util/Log:>d
("Ljava/lang/String;
Ljava/lang/String;)I
move-result v1
return-void
.end method
```

Inside methods you'll see instructions like:

- invoke-virtual
- move-result
- const-string
- return-void

These are bytecode instructions executed one by one.

INNER CLASSES

Inner classes use \$ in file names.

- MainActivity\$SettingsFragment.smali
- MainActivity\$1.smali
- MainActivity\$2.smali
- LoginActivity\$Callback.smali

Anonymous classes often get numeric names.

ACCESS MODIFIERS

public	Accessible from anywhere.
private	Accessible only within the class
protected	Accessible to subclasses
static	Belongs to the class (not object)
final	Cannot be overridden or modified
abstract	No implementation

Used in: .class public or .method private

HOW EVERYTHING FITS TOGETHER

```
.class
├── .super
├── .implements
├── .field
├── .method
├── Instructions
└── .end method
```

Think of every Smali file as a blueprint.

Once you recognize this pattern, reading Smali becomes much less intimidating.

BUG HUNTER PERSPECTIVE - WHY DEX STRUCTURE MATTERS

Metadata reveals valuable information before you read a single instruction.

String IDs	Method IDs	Class Definitions	Code Items
• Hidden API endpoints • Hardcoded credentials • Firebase URLs • Encryption keys • Debug messages • Feature flags	• Cryptographic functions • Authentication routines • Root detection methods • SSL pinning • Biometric auth logic	• Exported Activities • Internal Services • Utility classes • Network clients • Database managers	• Contain the actual instructions that instructing flow & determine how the application behaves.

Before reading Smali, know where to look. Structure reveals purpose.

PRACTICAL EXERCISE

Extract any APK using APKTool

```
$ apktool d app.apk
```

- 1 Locate classes.dex
- 2 Open APK in JADX
- 3 Browse the package structure
- 4 Observe how classes, methods, and fields map to DEX entries
- 5 Open matching Smali file and compare with decompiled Java

Compare & Observe:

- ✓ Same method?
- ✓ Variable names reconstructed?
- ✓ Control flow match?
- ✓ Which view is closer to the bytecode?

QUICK RECAP

- ✓ DEX is Android's executable format.
- ✓ classes.dex stores compiled logic.
- ✓ DEX uses indexed tables for efficiency.
- ✓ String IDs, Type IDs, Proto IDs, Field IDs & Method IDs describe the structure.
- ✓ Class Definitions organize classes.
- ✓ Code Items contain executable bytecode instructions.
- ✓ Understand DEX = Easier Smali analysis.

FINAL THOUGHT

Most people see an APK as a single file. Reverse engineers see a carefully organized blueprint.

T Every string has an index. ⚙ Every method has an identity. </> Every class has a definition. 010 0V0 Every instruction has a place.

Before you can read Android's language... You need to understand the book it's written in.

By - bithowl

When most people open a Smali file for the first time, they have the same reaction.

"What am I looking at?" There's no public class. No familiar Java syntax. No curly braces.

Instead, you see lines like: `.class public Lcom/example/app/MainActivity;`

`.super Landroidx/appcompat/app/AppCompatActivity;`

It feels like a completely different language.

But here's the secret: A Smali file isn't random. It's incredibly organized.

Every file follows almost the same structure.

Once you learn that structure, you can open almost any Android application and immediately understand what you're looking at.

Today, we're going to dissect a Smali file line by line.

The Story (Hook)

Imagine you're standing outside a house you've never visited before.

At first, it looks unfamiliar. But after stepping inside, you begin recognizing familiar rooms.

The entrance. The living room. The kitchen. The bedrooms.

Every house is different, yet almost all follow a similar layout.

Smali files work exactly the same way.

The names may change. The methods may change. The classes may change.

But the structure rarely does.

Once you understand the layout, navigating any Smali file becomes much easier.

A Typical Smali File

Most Smali files follow this general structure:

`.class`

`.super`

`.source`

`.implements`

`.field`

`.method`

...

.end method

.method

...

.end method

Each directive describes one part of the compiled class.

Let's examine them one by one.

1) .class — Defining the Class

Every Smali file begins with the .class directive.

Example: **.class public Lcom/example/app/LoginActivity;**

This line tells Android:

- This file represents a class.
- The class is public.
- Its fully qualified name is: com.example.app.LoginActivity

Notice something unusual: Lcom/example/app/LoginActivity;

This is called a **Type Descriptor**.

In Smali:

- L means **Object Type**
- / replaces Java's .
- ; marks the end of the type

So: Landroid/content/Intent;

is simply: android.content.Intent

2) .super — Parent Class

Every Java class extends another class.

Smali expresses this using: **.super Landroidx/appcompat/app/AppCompatActivity;**

Equivalent Java: extends AppCompatActivity

This tells Android which parent class should be inherited.

3).source — Original Source File

Example: .source "LoginActivity.java"

This records the source filename used during compilation.

Although useful for debugging, obfuscation tools often rename or remove this information.

4).implements — Implementing Interfaces

If a class implements interfaces, you'll see: `.implements Ljava/lang/Runnable;`

Equivalent Java: `implements Runnable`

A class may contain multiple `.implements` directives.

This helps Android understand which contracts the class fulfills.

5).field — Class Variables

Fields are variables that belong to a class.

Example: `.field private token:Ljava/lang/String;`

Equivalent Java: `private String token;`

Another example: `.field public static VERSION:I`

Equivalent Java: `public static int VERSION;`

Fields may include modifiers such as:

- `public`
- `private`
- `protected`
- `static`
- `final`
- `volatile`
- `transient`

These modifiers behave similarly to Java.

6).method — Defining a Method

Methods contain executable code.

Example: `.method public login()V`

Equivalent Java: `public void login()`

Everything between:

`.method`

and

`.end method`

belongs to that method.

This block contains:

- Register declarations

- Instructions
- Method calls
- Branches
- Return statements

Most of your reverse engineering work happens here.

7).end method

Every method must end with: .end method

This tells the assembler that the current method definition is complete.

Think of it like the closing brace (}) in Java.

8) Constructors

Constructors look slightly different.

Instead of using the class name, Smali uses: <init>

Example: .method public constructor <init>()V

Equivalent Java: public LoginActivity()

Static constructors use: <clinit>

These execute once when the class is first loaded.

You'll commonly find initialization code here.

Understanding Methods

Inside every method you'll eventually encounter instructions like:

invoke-virtual, move-result, const-string, return-void

Don't worry about memorizing them yet.

For now, simply recognize that every method is composed of bytecode instructions executed one after another.

We'll study these instructions in later chapters.

Inner Classes

Java allows classes to exist inside other classes.

Example: MainActivity.SettingsFragment

In Smali you'll often see: MainActivity\$SettingsFragment.smali

The \$ symbol indicates an inner or nested class.

Common examples include:

MainActivity\$1.smali

MainActivity\$2.smali

LoginActivity\$Callback.smali

Anonymous classes frequently receive numeric names.

These files are extremely common in Android applications.

Access Modifiers

Just like Java, Smali uses access modifiers to control visibility.

Common modifiers include:

1.Public: Accessible from anywhere

2.Private: Accessible only within the class

3.Protected: Accessible to subclasses

4.Static: Belongs to the class rather than an object

5.Final: Cannot be overridden or modified

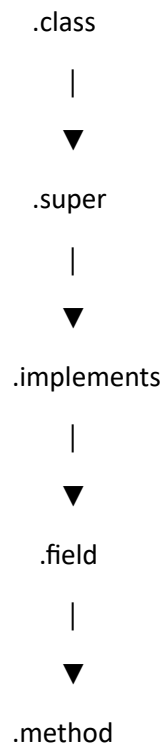
6.Abstract: Method or class has no implementation

These modifiers appear directly within directives such as: `.class public` or `.method private`

Understanding these modifiers helps identify sensitive methods, hidden APIs, and internal application logic.

How Everything Fits Together

Think of every Smali file as a blueprint.



|



Instructions

|



.end method

Once you recognize this pattern, reading Smali becomes much less intimidating.

Bug Hunter Perspective

Experienced Android security researchers rarely read Smali from top to bottom.

Instead, they scan the file structure to quickly understand what they're dealing with.

Questions they ask include:

- Is this an Activity, Service, Broadcast Receiver, or utility class?
- Which class does it extend?
- Does it implement security-sensitive interfaces?
- Are there suspicious static fields storing secrets?
- Which methods look related to authentication, encryption, networking, or root detection?
- Are there inner classes handling callbacks or background tasks?

By recognizing these structural clues, researchers can quickly prioritize which files deserve deeper analysis.

For example:

- A class extending WebViewClient may contain WebView security logic.
- A class implementing X509TrustManager often relates to SSL certificate validation.
- Fields named API_KEY, SECRET, or BASE_URL may reveal sensitive configuration.
- Constructors can expose initialization routines and dependency injection.

Before reading a single bytecode instruction, the file's structure often reveals its purpose.

Practical Exercise

Extract any APK using APKTool: `apktool d app.apk`

Open any .smali file and try to identify:

- The class name (.class)
- The parent class (.super)
- The source filename (.source)

- Any implemented interfaces (.implements)
- All declared fields (.field)
- Constructors (<init> or <clinit>)
- Public and private methods
- Inner classes (files containing \$)

Don't worry about understanding every instruction.

For now, focus on recognizing the overall layout.

Once you can identify these building blocks instantly, learning Smali instructions becomes much easier.

Quick Recap

- Every Smali file follows a predictable structure.
- .class defines the class and its access level.
- .super identifies the parent class.
- .source records the original source filename.
- .implements lists implemented interfaces.
- .field declares class variables.
- .method begins a method definition, while .end method closes it.
- Constructors use <init>, and static initializers use <clinit>.
- Inner classes use the \$ naming convention.
- Understanding file structure is the first step toward reading Smali efficiently.

Final Thought

A beginner opens a Smali file and sees strange symbols.

An experienced reverse engineer opens the same file and immediately sees a class hierarchy, object relationships, fields, methods, and potential attack surfaces.

The code hasn't changed. Only the way it's being read.

Because before you understand what a program **does**, you must first understand how it's **organized**.

By - bithowl